Air University

Department of Computer Science

CS426L Introduction to Parallel & Distributed Computing

Lab 07 Report: OpenMP II, Scheduling & Reduction

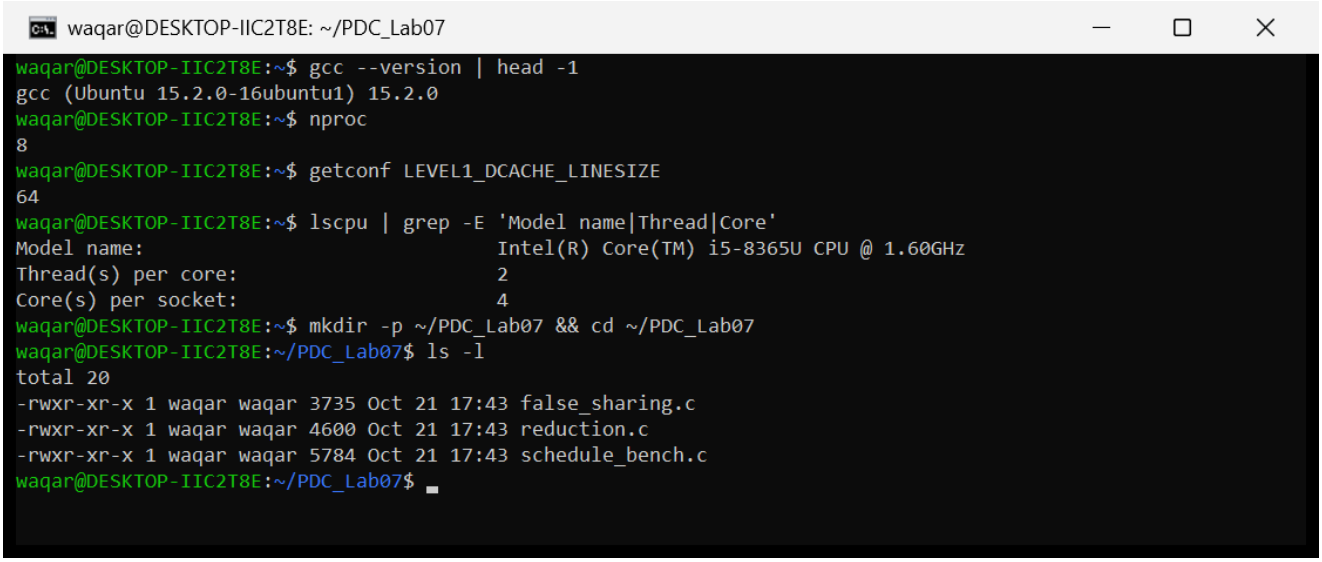
Item	Details
Student	Waqar
Instructor	Muhammad Nauman Zafar Hashmi
Semester	VII C (FALL26)
Environment	WSL 2 (Ubuntu), GCC 15.2.0 with OpenMP (libgomp), flags -O2 -fopenmp -std=gnu11
Machine	Intel Core i5-8365U @ 1.60 GHz, 4 cores / 8 logical CPUs (hyper-threading), laptop
Logical cores (nproc)	8
Cache line (getconf LEVEL1_DCACHE_LINESIZE)	64 bytes
Date	21 October 2026

Note on measurements. All numbers in this report are copied from the terminal output shown in the screenshots (the raw output is also in the *raw_output* folder). The machine is a laptop, so the clock speed changes with temperature and background load; single runs vary by about 10-15%. I kept the numbers as measured and point out where noise affects them.

Before you start

I opened a WSL terminal, checked the compiler, the number of logical cores and the cache-line size, and created `~/PDC_Lab07`. Logical cores: **8**. Cache line: **64 bytes**.

Screenshot 1: gcc --version, nproc, getconf LEVEL1_DCACHE_LINESIZE, lscpu and the lab folder.



Task 1: Parallel Sum and Max: reduction vs critical

The program sums and finds the maximum of 5,000,000 integers (values 0 to 99,999, generated with a fixed seed so every run uses the same data). Five versions are timed, each 5 times, and the average is printed. Correctness is checked against the serial result in every repetition. A short warm-up is run first so the CPU clock is up before the first timed version.

Variants with 4 threads

Variant	Sum correct? (Y/N)	Max correct? (Y/N)	Average time (ms)
serial	Y	Y	4.184
racy	N	Y	3.574
critical-each	Y	Y	1001.362
critical-partial	Y	Y	1.573
reduction	Y	Y	2.502

Scaling (average time in ms)

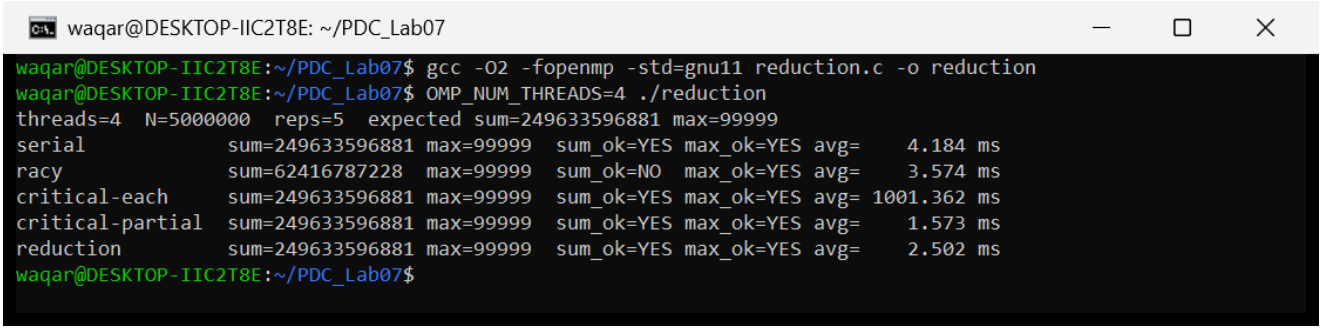
Threads	critical-each	critical-partial	reduction	Reduction speedup
1	165.668	6.388	7.792	1.00
2	408.858	4.049	4.042	1.93
4	743.372	1.963	2.793	2.79
8	1089.721	2.503	1.503	5.18

Reduction speedup = time with 1 thread / time with t threads (7.792 / t-thread time). The loop only does an add and a compare per element, so it is limited by memory bandwidth and the times are only a few milliseconds; that is why the speedup does not grow smoothly (the 8-thread value is helped by noise in such short runs). critical-each gets slower with every thread added.

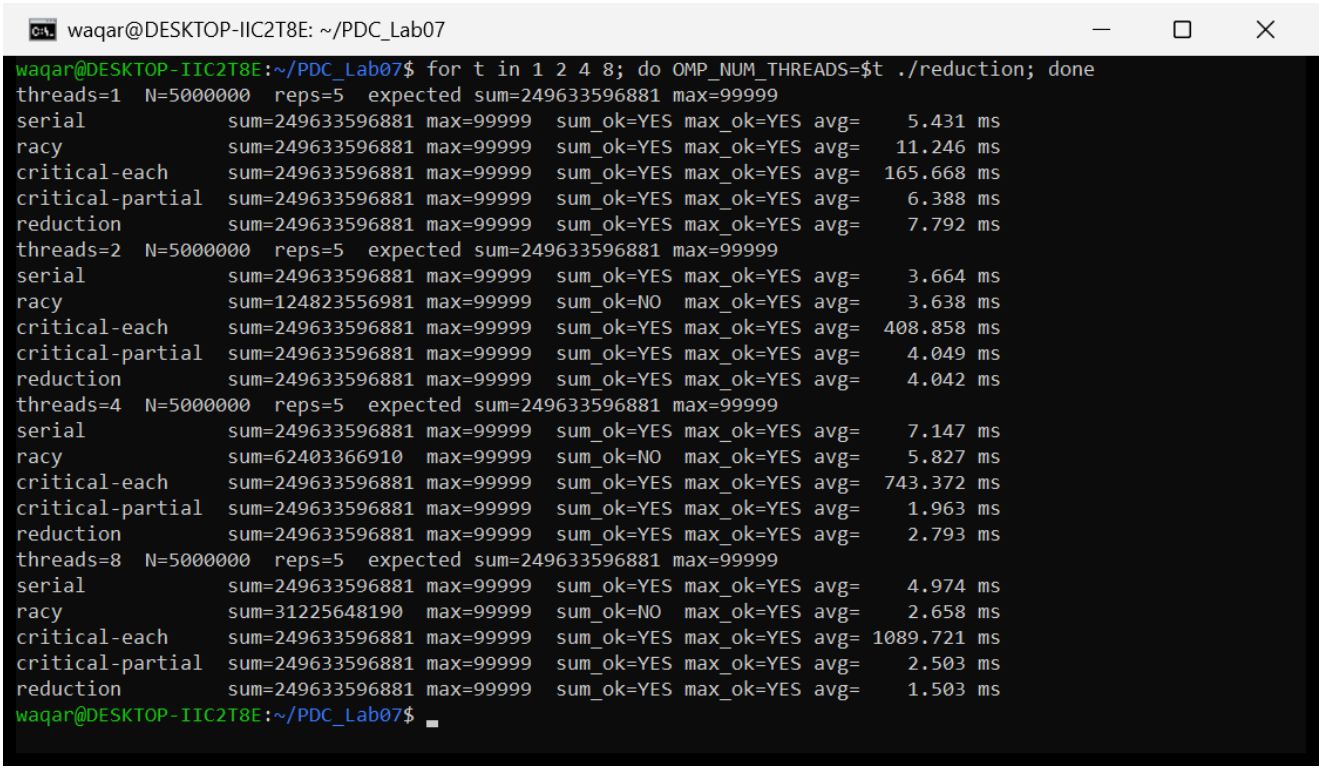
Observation of the racy version

With 4 threads the racy sum was **62,416,787,228** in the first run and again 62,416,787,228 in the two extra runs, but 62,403,366,910 in the 4-thread run of the scaling loop. So the wrong value was often the same but not always. The correct sum is 249,633,596,881, so the racy result is almost exactly one quarter of it. With 2 threads it was 124,823,556,981 (about one half) and with 8 threads 31,225,648,190 (about one eighth). With 1 thread the racy version is correct, because there is nobody to race with. The maximum (99,999) was correct in every run.

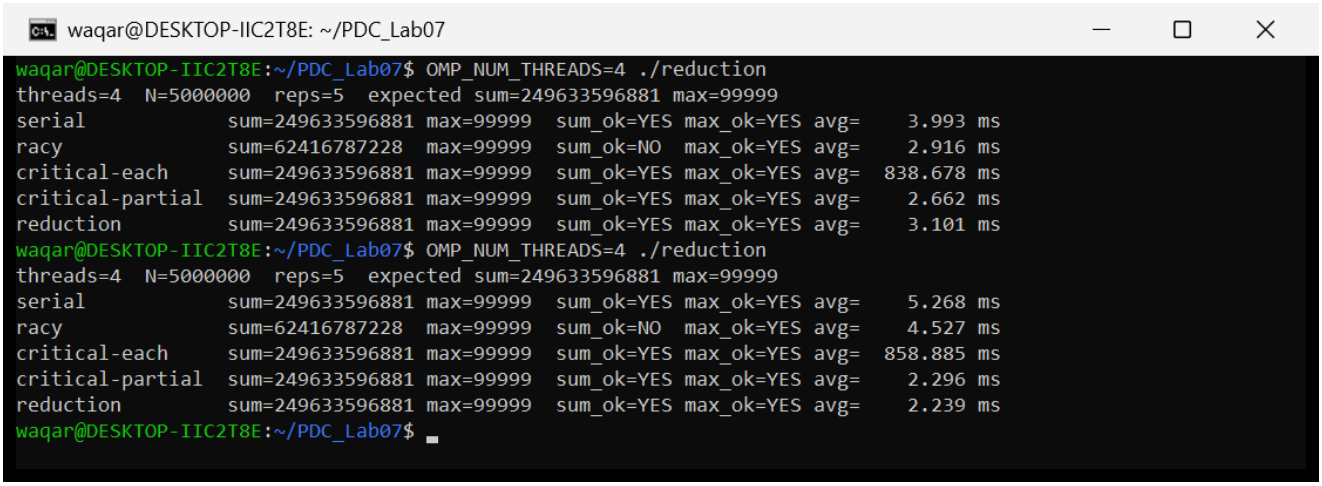
Screenshot 2: building reduction.c and running it with 4 threads.



Screenshot 3: scaling run with 1, 2, 4 and 8 threads.



Screenshot 4: two more runs with 4 threads to check the racy sum.



Discussion questions

Q1. Why does the racy version give a wrong sum, and was the wrong value the same in every run? Why is the maximum often still correct?

`sum += data[i]` is a read-modify-write on a shared variable, and nothing stops two threads from reading the same old value and both writing back, so updates are lost. In my build -O2 even keeps `sum` in a register inside each thread and writes it back at the end, so each thread overwrites the others and the result is basically one thread's partial sum. That is why the answer was about 1/4 of the real sum with 4 threads, 1/2 with 2 and 1/8 with 8. The value was the same in three of the four 4-thread runs (62,416,787,228) and different once (62,403,366,910): it depends on which thread wrote last, which is up to timing. The maximum survives because it is idempotent and the largest value (99,999) appears many times in every thread's chunk, so whichever thread writes last still writes 99,999. A lost max update only matters if the true maximum appeared in just one place and that write was lost.

Q2. Why is the critical-each version so much slower than the serial loop, and why does it get no faster (or even slower) with more threads?

It enters and leaves a lock 5 million times. Each entry costs far more than the add it protects, so even with 1 thread it took 165.7 ms against about 4-7 ms for the serial loop. With more threads only one thread can be inside at a time, so the work is fully serialized, and on top of that the threads fight over the lock and the cache line that holds it and the shared variables, which keeps moving between cores. The time grew from 165.7 ms (1 thread) to 408.9, 743.4 and 1089.7 ms with 2, 4 and 8 threads.

Q3. Explain how the reduction clause works: what does each thread start with, and when are the private copies combined? What are the identity values for + and for max?

For each variable in the clause, every thread gets its own private copy, initialized to the identity value of the operator. Each thread updates only its copy during the loop, with no synchronization. At the end of the construct OpenMP combines the private copies, together with the original value, using the operator, and stores the result in the shared variable. The identity for + is 0. The identity for max is the smallest value of the type (INT_MIN for int), so any real value replaces it.

Q4. Compare critical-partial with reduction. Do they do the same thing, and why is the clause preferable to writing it by hand?

Yes, they do the same thing: private partial results, then one combination per thread. That is why their times are close (1.57 ms and 2.50 ms with 4 threads; the difference is noise in a 2 ms measurement). The clause is preferable because it is one line instead of extra variables, a critical section and a merge, so it is harder to get wrong (wrong initial value, forgetting a variable, wrong merge). The runtime can also combine the copies in a better way, for example as a tree instead of one at a time.

Q5. The program uses integers. Why could a floating-point sum computed with a reduction differ slightly between runs or thread counts, even though the program is correct?

Floating-point addition is not associative: (a+b)+c can round differently from a+(b+c). A reduction changes the order of additions: each thread sums its own chunk and the partial sums are combined in some order. With a different number of threads the chunks change, and the combination order may differ between runs, so the last bits of the result can change. Integer addition is exact (as long as it does not overflow), so the integer result is always the same.

Source code: reduction.c

```
/* reduction.c - Task 1: parallel sum and max, reduction clause vs critical
   build: gcc -O2 -fopenmp -std=gnu11 reduction.c -o reduction
   run:   OMP_NUM_THREADS=4 ./reduction */
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

#define N       5000000     /* number of array elements */
#define REPS    5           /* timed repetitions per version, average is printed */
#define MAX_VAL 100000      /* values are in 0 .. MAX_VAL-1 */

int *data;
long long ref_sum;          /* filled by the serial version */
int ref_max;

/* fill the array with pseudo random values (same values on every run) */
void init_data()
{
    unsigned int x = 12345;
    for (int i = 0; i < N; i++) {
        x = x * 1103515245u + 12345u;
        data[i] = (x >> 8) % MAX_VAL;
    }
}

void serial_version(long long *sum_out, int *max_out)
{
    long long sum = 0;
    int max_val = 0;
    for (int i = 0; i < N; i++) {
        sum += data[i];
        if (data[i] > max_val)
            max_val = data[i];
    }
    *sum_out = sum;
    *max_out = max_val;
}
```

```
/* wrong on purpose: every thread updates the shared sum and max without protection */
void racy_version(long long *sum_out, int *max_out)
{
    long long sum = 0;
    int max_val = 0;
    #pragma omp parallel for
    for (int i = 0; i < N; i++) {
        sum += data[i];
        if (data[i] > max_val)
            max_val = data[i];
    }
    *sum_out = sum;
    *max_out = max_val;
}

/* correct but slow: one critical section for every single element */
void critical_each_version(long long *sum_out, int *max_out)
{
    long long sum = 0;
    int max_val = 0;
    #pragma omp parallel for
    for (int i = 0; i < N; i++) {
        #pragma omp critical
        {
            sum += data[i];
            if (data[i] > max_val)
                max_val = data[i];
        }
    }
    *sum_out = sum;
    *max_out = max_val;
}

/* each thread works on private partial results, one critical section per thread at the end */
void critical_partial_version(long long *sum_out, int *max_out)
{
    long long sum = 0;
    int max_val = 0;
    #pragma omp parallel
    {
        long long my_sum = 0;
        int my_max = 0;
        #pragma omp for
        for (int i = 0; i < N; i++) {
            my_sum += data[i];
            if (data[i] > my_max)
                my_max = data[i];
        }
        #pragma omp critical
        {
            sum += my_sum;
            if (my_max > max_val)
                max_val = my_max;
        }
    }
    *sum_out = sum;
    *max_out = max_val;
}

/* same idea as critical-partial, but OpenMP creates and combines the private copies */
void reduction_version(long long *sum_out, int *max_out)
{
    long long sum = 0;
    int max_val = 0;
    #pragma omp parallel for reduction(+:sum) reduction(max:max_val)
    for (int i = 0; i < N; i++) {
        sum += data[i];
        if (data[i] > max_val)
            max_val = data[i];
    }
    *sum_out = sum;
    *max_out = max_val;
}

typedef void (*version_fn)(long long *, int *);

void run_version(const char *name, version_fn fn)
{
    long long sum = 0;
    int max_val = 0;
    double total = 0.0;
    int sum_ok = 1, max_ok = 1;

    for (int r = 0; r < REPS; r++) {
```

```
        double t0 = omp_get_wtime();
        fn(&sum, &max_val);
        total += omp_get_wtime() - t0;
        if (sum != ref_sum) sum_ok = 0;     /* must be right in every repetition */
        if (max_val != ref_max) max_ok = 0;
    }
    printf("%-17s sum=%-12lld max=%-6d sum_ok=%-3s max_ok=%-3s avg=%9.3f ms\n",
           name, sum, max_val, sum_ok ? "YES" : "NO", max_ok ? "YES" : "NO",
           total / REPS * 1000.0);
}

int main()
{
    data = malloc(N * sizeof(int));
    if (data == NULL) {
        printf("malloc failed\n");
        return 1;
    }
    init_data();
    serial_version(&ref_sum, &ref_max);     /* reference values */

    /* warm-up (not timed): creates the thread team and lets the CPU clock up,
       otherwise the first version measured (serial) looks slower than it is */
    long long tmp_sum;
    int tmp_max;
    for (int r = 0; r < 30; r++) {
        serial_version(&tmp_sum, &tmp_max);
        reduction_version(&tmp_sum, &tmp_max);
    }

    printf("threads=%d  N=%d  reps=%d  expected sum=%lld max=%d\n",
           omp_get_max_threads(), N, REPS, ref_sum, ref_max);
    run_version("serial", serial_version);
    run_version("racy", racy_version);
    run_version("critical-each", critical_each_version);
    run_version("critical-partial", critical_partial_version);
    run_version("reduction", reduction_version);

    free(data);
    return 0;
}
```

Task 2: Benchmarking Static, Dynamic and Guided Scheduling

The loop has N = 4000 iterations and iteration i costs (i+1) x 20 steps of a floating-point recurrence, so the last iteration is 4000 times more expensive than the first. The schedule is chosen with `schedule(runtime)` and `omp_set_schedule()`. With `nowait` each thread records how long it was busy in its own iterations. Each policy is run 3 times and the best run is kept; imbalance = busy max / busy min. Results are checked against a serial run (best of 5). `work()` is marked noinline so the serial and parallel loops run the same machine code.

Uneven workload with 4 threads (best of 3 runs)

Policy	Wall time (ms)	Busy max (ms)	Busy min (ms)	Imbalance	Correct? (Y/N)
static	292.04	291.71	40.62	7.18	Y
static,1	152.86	152.86	151.32	1.01	Y
static,64	257.99	257.78	239.57	1.08	Y
dynamic,1	260.94	260.84	260.56	1.00	Y
dynamic,64	187.46	187.16	176.32	1.06	Y
guided,1	182.19	182.13	181.79	1.00	Y
guided,64	189.69	189.64	180.74	1.05	Y

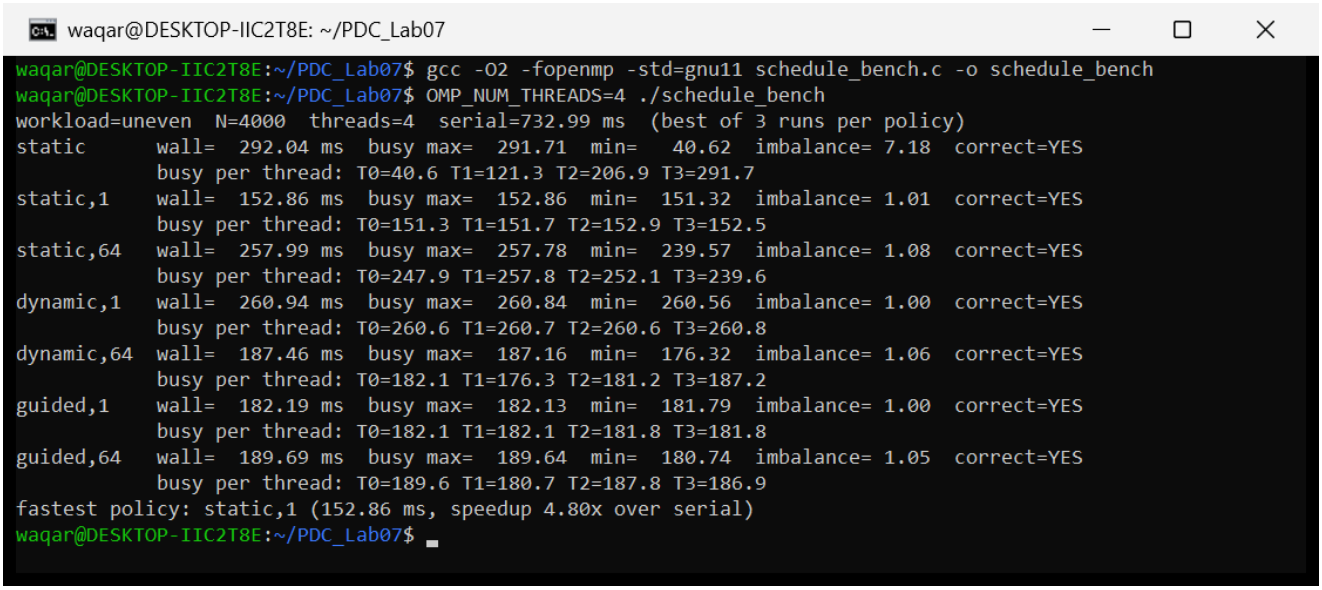
Number of logical cores (nproc)	Serial time (ms)	Fastest policy at 2 / 4 / 8 threads
8	732.99	static,1 / static,1 / guided,64

Per-thread busy times for plain static with 4 threads: T0 = 40.6, T1 = 121.3, T2 = 206.9, T3 = 291.7 ms. The static,64 and dynamic,1 rows are slower than the other balanced policies even though their imbalance is about 1.0. All threads were slowed down equally in those runs, which points to the laptop's clock speed dropping rather than to the schedule. In my earlier test runs both of them were around 185-190 ms. The speedup of 4.80x printed by the program is above 4 for the same reason: the serial time is a little inflated by noise. The useful comparison is between the policies.

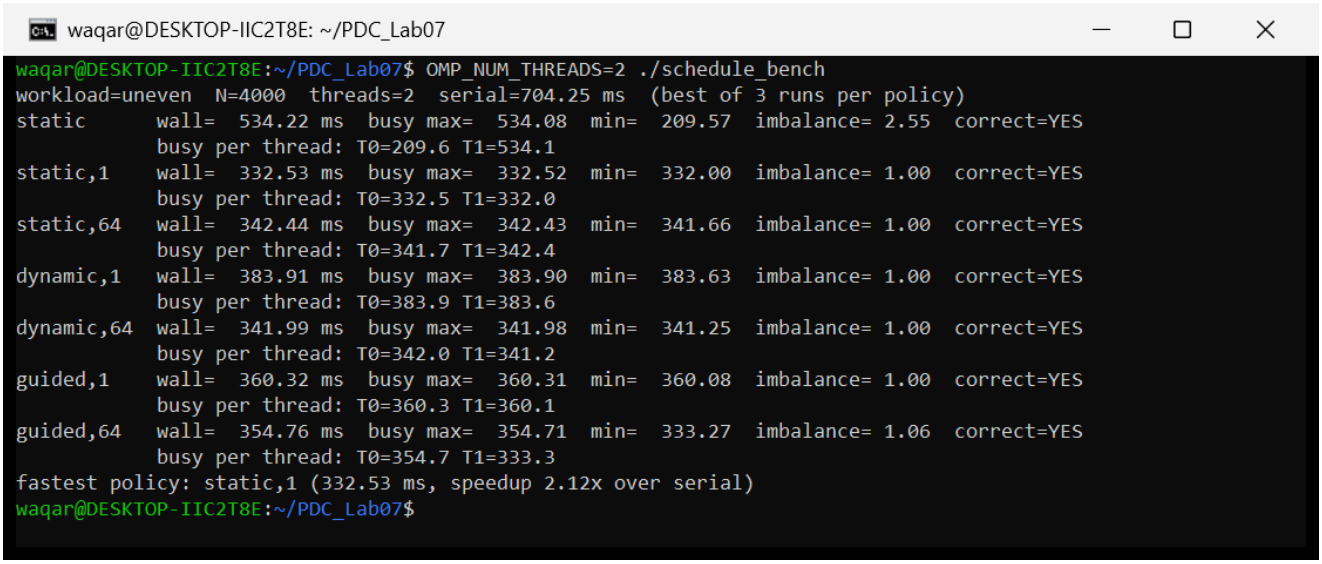
2 and 8 threads (uneven workload)

Threads	static wall (ms)	static imbalance	Fastest policy	Fastest wall (ms)
2	534.22	2.55	static,1	332.53
4	292.04	7.18	static,1	152.86
8	186.99	13.36	guided,64	104.81

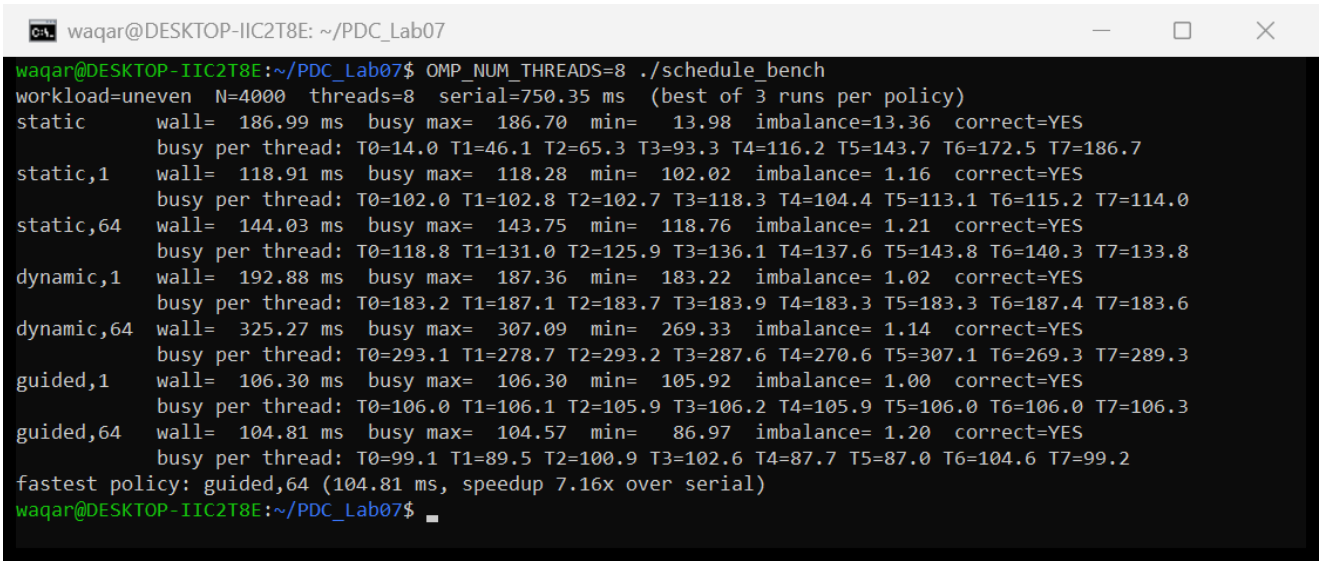
Screenshot 5: building schedule_bench.c and running it with 4 threads (uneven workload).



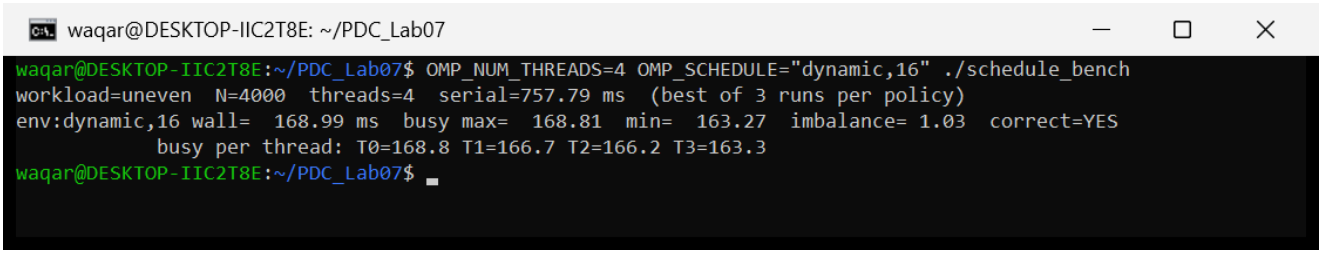
Screenshot 6: uneven workload with 2 threads.



Screenshot 7: uneven workload with 8 threads.



Screenshot 8 (optional step): one policy from the environment, OMP_SCHEDULE="dynamic,16" (wall 159 ms, imbalance 1.02).



Discussion questions

Q1. Why does the plain static policy show a large imbalance on this workload? Identify which thread (or worker) is busy longest and explain why.

Plain static splits the 4000 iterations into 4 contiguous blocks of 1000. Because the cost grows with i, the blocks are very different in size: T0 gets iterations 0-999 (the cheapest) and T3 gets 3000-3999 (the most expensive). T3 was busy longest (291.7 ms) and T0 shortest (40.6 ms), so T0 sat idle for most of the run and the wall time is set by T3. The block costs grow roughly as 1 : 3 : 5 : 7, which is close to the measured 40.6 : 121.3 : 206.9 : 291.7.

Q2. Explain why static,1, the dynamic policies and the guided policies balance the work much better, and what each of them pays for it.

static,1 deals the iterations out round-robin, so every thread gets cheap and expensive iterations and the totals are almost equal (imbalance 1.01). It costs nothing at run time, but it only works because the cost grows smoothly; it also gives poor locality for neighbouring data. Dynamic hands out chunks on demand, so a thread that finishes early simply takes more work. It adapts to any pattern but pays for a shared counter (an atomic operation) on every chunk, which is most expensive with chunk 1. Guided also works on demand but starts with large chunks and shrinks them, so it needs far fewer chunk requests than dynamic,1 while still balancing the end of the loop. It pays a little for computing the chunk size and can still be uneven if a big early chunk happens to be expensive.

Q3. How did the results change with 2 and with 8 threads? When the number of threads is larger than the number of cores, what does the busy time of a thread really measure?

With 2 threads the imbalance of static was smaller (2.55, blocks 1 : 3) and static,1 was fastest (332.5 ms). With 8 threads the imbalance of static grew to 13.36 (T0 14.0 ms vs T7 186.7 ms), because each block is smaller and the first and last blocks differ more. The best policy was guided,64 (104.8 ms), with guided,1 and static,1 close. 8 threads is the number of logical CPUs but only 4 are real cores, so two threads share a core through hyper-threading. Busy time is wall-clock time between the start and the end of a thread's iterations, so it also counts time when the thread was waiting for its core or sharing it. With more threads than cores it measures time in the loop, not time actually computing.

Q4. What is the effect of the chunk size? Compare chunk size 1 with chunk size 64 for the dynamic policy, and name the trade-off.

A small chunk gives the best balance: dynamic,1 had imbalance 1.00 with every thread finishing within 0.3 ms, but each iteration needs a trip to the shared counter. Chunk 64 means 64 times fewer requests, but the last chunks are big and expensive, so threads finish at more different times (imbalance 1.06 with 4 threads, 1.14 with 8). In my 4-thread run dynamic,64 was faster (187.5 vs 260.9 ms), although part of that gap is the clock-speed noise mentioned above. The trade-off is scheduling overhead versus load balance: small chunks balance better but cost more per iteration, large chunks are cheap but balance worse.

Q5. Why does guided start with large chunks and then shrink them? In what kind of workload would that be an advantage over dynamic,1?

Each guided chunk is about (remaining iterations / number of threads), so the first chunks are large and the overhead is low while there is a lot of work left, and the chunks get small near the end, where balance matters, so nobody is left holding a big last chunk. This beats dynamic,1 when there are many cheap

iterations, where dynamic,1 spends a noticeable fraction of its time fetching iterations one at a time and fighting over the counter, and when the cost is irregular but not heavily concentrated at the start of the loop.

Source code: schedule_bench.c

```
/* schedule_bench.c - Task 2 and 4: static, dynamic and guided scheduling
   build: gcc -O2 -fopenmp -std=gnu11 schedule_bench.c -o schedule_bench
          gcc -O2 -fopenmp -std=gnu11 -DUNIFORM schedule_bench.c -o schedule_bench_uniform
   run:   OMP_NUM_THREADS=4 ./schedule_bench
          OMP_NUM_THREADS=4 OMP_SCHEDULE="dynamic,16" ./schedule_bench   (one policy only) */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <omp.h>

#define N         4000      /* loop iterations */
#define COST_STEP 20        /* uneven: iteration i costs (i+1)*COST_STEP steps */
#define RUNS      3         /* each policy is run 3 times, the best run is kept */
#define SERIAL_RUNS 5       /* the single-thread time is noisier on a laptop, so best of 5 */
#define MAX_THREADS 64

double result[N];
double expected[N];
double busy[MAX_THREADS];   /* time each thread spent inside its iterations */

/* how many inner steps iteration i needs */
long cost(int i)
{
#ifdef UNIFORM
    (void)i;                                /* every iteration costs the same */
    return (long)(N + 1) / 2 * COST_STEP;   /* same total work as the uneven case */
#else
    return (long)(i + 1) * COST_STEP;       /* later iterations are much more expensive */
#endif
}

/* the work of one iteration, the result depends only on i.
   noinline so the serial loop and the parallel loop run exactly the same machine code */
__attribute__((noinline)) double work(int i)
{
    long steps = cost(i);
    double x = i * 0.001;
    for (long k = 0; k < steps; k++)
        x = x * 0.999999 + (double)(k & 7);
    return x;
}

typedef struct {
    const char *name;
    omp_sched_t kind;
    int chunk;              /* 0 means the default chunk size */
} policy_t;

/* runs the loop once with the schedule set at run time, returns the wall time in ms */
double run_once(int *nthreads)
{
    double start = omp_get_wtime();
    #pragma omp parallel
    {
        int tid = omp_get_thread_num();
        double t0 = omp_get_wtime();
        #pragma omp for schedule(runtime) nowait
        for (int i = 0; i < N; i++)
            result[i] = work(i);
        busy[tid] = (omp_get_wtime() - t0) * 1000.0;
        #pragma omp single
        *nthreads = omp_get_num_threads();
    }
    return (omp_get_wtime() - start) * 1000.0;
}

int check_result()
{
    for (int i = 0; i < N; i++)
        if (result[i] != expected[i])
            return 0;
    return 1;
}

/* runs one policy RUNS times and prints the best run */
double bench_policy(const char *name, int print_threads)
{
```

```
    double best_wall = 1e30, best_busy[MAX_THREADS];
    int nthreads = 1, correct = 1;

    for (int r = 0; r < RUNS; r++) {
        memset(result, 0, sizeof(result));
        double wall = run_once(&nthreads);
        if (!check_result())
            correct = 0;
        if (wall < best_wall) {
            best_wall = wall;
            memcpy(best_busy, busy, sizeof(busy));
        }
    }

    double bmax = best_busy[0], bmin = best_busy[0];
    for (int t = 1; t < nthreads; t++) {
        if (best_busy[t] > bmax) bmax = best_busy[t];
        if (best_busy[t] < bmin) bmin = best_busy[t];
    }
    printf("%-11s wall=%8.2f ms  busy max=%8.2f  min=%8.2f  imbalance=%5.2f  correct=%s\n",
           name, best_wall, bmax, bmin, bmax / bmin, correct ? "YES" : "NO");
    if (print_threads) {
        printf("            busy per thread:");
        for (int t = 0; t < nthreads; t++)
            printf(" T%d=%.1f", t, best_busy[t]);
        printf("\n");
    }
    return best_wall;
}

int main()
{
    policy_t policies[] = {
        {"static",     omp_sched_static,  0},
        {"static,1",   omp_sched_static,  1},
        {"static,64",  omp_sched_static,  64},
        {"dynamic,1",  omp_sched_dynamic, 1},
        {"dynamic,64", omp_sched_dynamic, 64},
        {"guided,1",   omp_sched_guided,  1},
        {"guided,64",  omp_sched_guided,  64},
    };
    int npolicies = sizeof(policies) / sizeof(policies[0]);

    /* warm-up run (not timed): creates the thread team and lets the CPU clock up,
       otherwise the first policy measured looks slower than it really is */
    int warm_threads, env_chunk;
    omp_sched_t env_kind;
    omp_get_schedule(&env_kind, &env_chunk);    /* keep what OMP_SCHEDULE asked for */
    omp_set_schedule(omp_sched_dynamic, 1);
    run_once(&warm_threads);
    omp_set_schedule(env_kind, env_chunk);

    /* serial reference: correct results and the time to beat (best of SERIAL_RUNS) */
    double serial_ms = 1e30;
    for (int r = 0; r < SERIAL_RUNS; r++) {
        double t0 = omp_get_wtime();
        for (int i = 0; i < N; i++)
            expected[i] = work(i);
        double ms = (omp_get_wtime() - t0) * 1000.0;
        if (ms < serial_ms)
            serial_ms = ms;
    }

#ifdef UNIFORM
    const char *workload = "uniform";
#else
    const char *workload = "uneven";
#endif
    printf("workload=%s  N=%d  threads=%d  serial=%.2f ms  (best of %d runs per policy)\n",
           workload, N, omp_get_max_threads(), serial_ms, RUNS);

    /* optional part: if OMP_SCHEDULE is set, only run that policy from the environment */
    char *env = getenv("OMP_SCHEDULE");
    if (env != NULL) {
        char label[64];
        snprintf(label, sizeof(label), "env:%s", env);
        bench_policy(label, 1);
        return 0;
    }

    int best = 0;
    double best_ms = 1e30;
    for (int p = 0; p < npolicies; p++) {
        omp_set_schedule(policies[p].kind, policies[p].chunk);
```

```
        double ms = bench_policy(policies[p].name, 1);
        if (ms < best_ms) {
            best_ms = ms;
            best = p;
        }
    }
    printf("fastest policy: %s (%.2f ms, speedup %.2fx over serial)\n",
           policies[best].name, best_ms, serial_ms / best_ms);
    return 0;
}
```

Task 3: False Sharing and Padding

Each thread counts how often bit 16 of its own random sequence is set, over 50,000,000 iterations. **adjacent** keeps the counters in a plain `long` array, so 8 counters share one 64-byte line. **padded** puts each counter in a 64-byte struct aligned to 64 bytes. **local-accumulate** counts in a private variable and writes the result once at the end. The shared counters are `volatile`, because otherwise -O2 keeps them in a register and the effect disappears. Each version is timed 3 times and the average is printed. The counters are checked against a serial computation.

Three runs with 4 threads

Run	adjacent (ms)	padded (ms)	local-accumulate (ms)	Counters correct? (Y/N)
1	391.13	199.07	133.13	Y
2	375.72	264.46	193.56	Y
3	369.27	238.41	141.06	Y

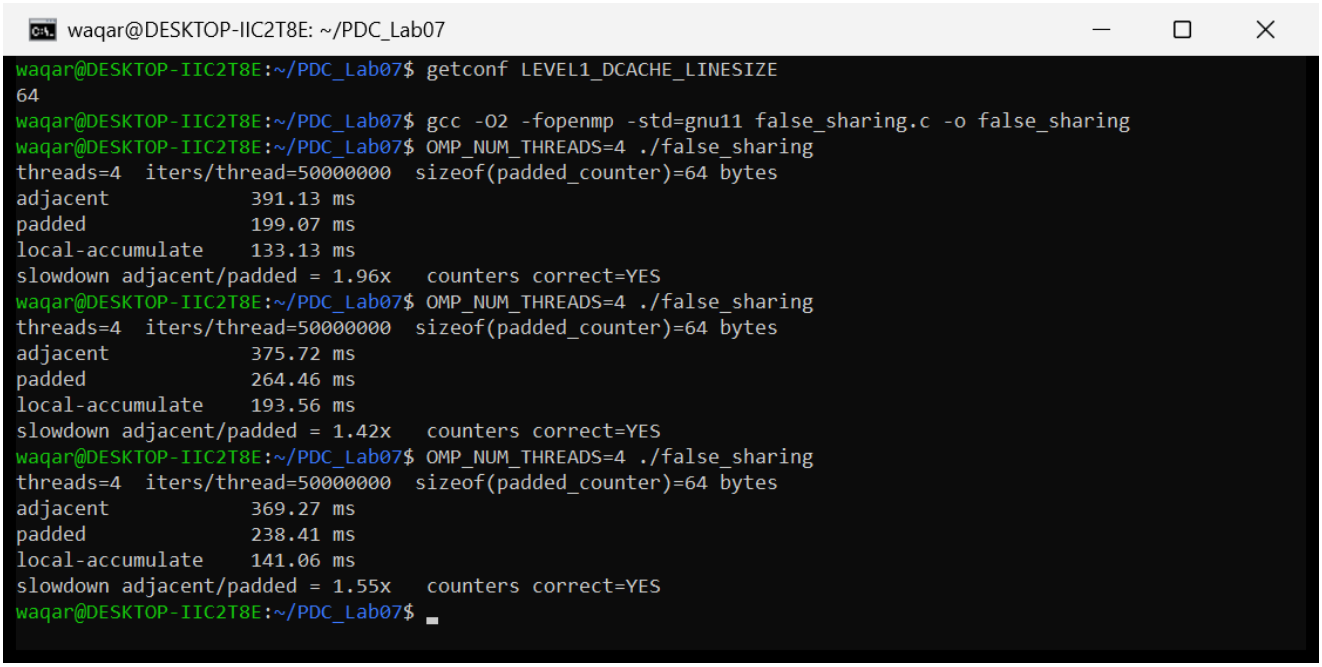
Effect of the number of threads

Threads	adjacent (ms)	padded (ms)	Slowdown adjacent / padded
1	174.28	167.72	1.04x
2	224.59	225.83	0.99x
4	388.22	203.03	1.91x
8	841.53	325.73	2.58x

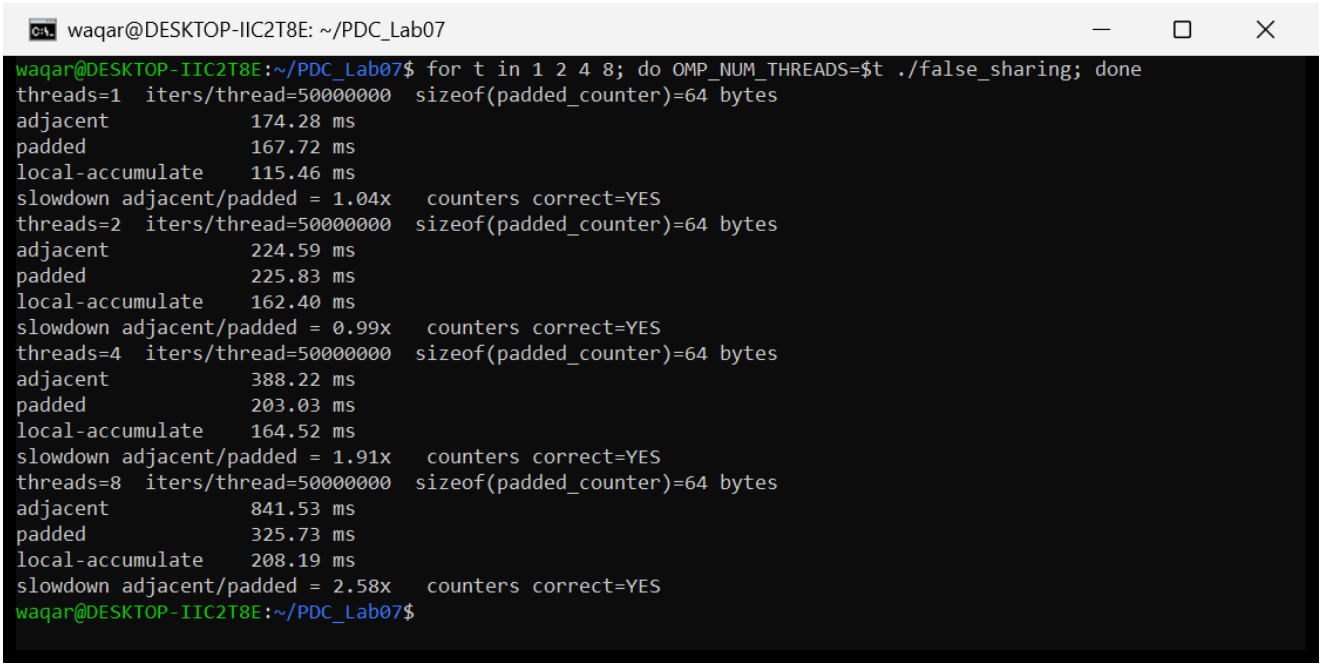
Cache-line size (bytes)	Number of logical cores (nproc)
64	8

Explanation of the effect. The adjacent counters are separate variables, but they sit in the same 64-byte cache line. Every increment needs that line in the core's cache in exclusive (Modified) state, so each write invalidates the copies in the other cores, and the line keeps moving between them. With 4 threads this made the same work about 2x slower (1.42x-1.96x over the three runs), and with 8 threads 2.58x. The 2-thread result (0.99x) needs a comment: Linux most likely put the two threads on the two hyper-threads of one physical core. Those share the same L1 cache, so there is no invalidation between cores and no false sharing to see.

Screenshot 9: cache-line size, building false_sharing.c and three runs with 4 threads.



Screenshot 10: false sharing with 1, 2, 4 and 8 threads.



Discussion questions

Q1. Why is the program with adjacent counters correct but slower, although no two threads ever write to the same variable? Explain in terms of cache lines and invalidation.

It is correct because every thread only touches its own element, so there is no data race and no lost update. It is slower because caches work with whole 64-byte lines, not single variables. Eight 8-byte counters fit in one line. When thread 0 writes its counter, the coherence protocol (MESI) invalidates that line in every other core, so the next write by thread 1 misses and has to fetch the line back. The line keeps moving between cores on almost every increment. This is false sharing: the hardware treats the line as shared even though the data is not.

Q2. Why does padding fix the problem, and what does it cost? Why was the effect absent or small with 1 thread?

Padding makes each counter fill a whole 64-byte line (sizeof(padded_counter) = 64, aligned to 64), so no two threads ever write the same line and each core keeps its own line in its cache. The cost is memory: 64 bytes per counter instead of 8 (8 times more), and those lines take up cache space. With 1 thread there is no other core to invalidate the line, so adjacent and padded behave the same (174.3 vs 167.7 ms, 1.04x).

Q3. Which of the three versions is fastest? Explain why local accumulation is the best remedy in practice, and when padding is still the right tool.

local-accumulate was fastest in every run (133-194 ms with 4 threads, against 199-264 ms for padded and 369-391 ms for adjacent). The counter lives in a register, so the loop does no memory write at all, and there is only one shared write per thread at the end. It removes the problem completely and also saves the store traffic. Padding is still the right tool when the per-thread data really has to be in shared memory while the loop runs, for example statistics or progress counters that another thread reads, per-thread queues or locks, or when the code that updates them cannot be restructured to use a local variable.

Q4. How did the slowdown change as the number of threads grew, and what would you expect if the threads ran on a single core?

It grew with the number of threads: 1.04x (1 thread), 0.99x (2), 1.91x (4) and 2.58x (8). More threads writing the same line means more invalidations and more cores waiting for the line. (With 2 threads the two threads were probably on one core, as explained above.) If all threads ran on a single core, they would share one L1 cache, so the line would never be invalidated and false sharing would disappear. The times would then be about the same for adjacent and padded, and both would be roughly the 1-thread time multiplied by the number of threads.

Q5. How would you recognize false sharing in a real program, where the counters are not so obvious? Name one symptom or tool.

The typical symptom is a program that gets slower, or stops scaling, when threads are added, even though the threads work on separate data and have no locks. Small per-thread structs, arrays indexed by thread id and fields of one struct written by different threads are common causes. Tools: `perf c2c` on Linux shows the cache lines with heavy contention (HITM events) and which code touches them; `perf stat` shows many cache misses, and Intel VTune has a memory-access analysis that points at false sharing.

Source code: false_sharing.c

```
/* false_sharing.c - Task 3: false sharing with adjacent counters, fixed with padding
   build: gcc -O2 -fopenmp -std=gnu11 false_sharing.c -o false_sharing
   run:   OMP_NUM_THREADS=4 ./false_sharing */
#include <stdio.h>
#include <omp.h>

#define ITERS       50000000L   /* loop iterations per thread */
#define REPS        3           /* timed repetitions per version, average is printed */
#define MAX_THREADS 64
#define CACHE_LINE  64          /* from: getconf LEVEL1_DCACHE_LINESIZE */

/* volatile so that -O2 really writes the counter to memory in every iteration
   (otherwise the compiler keeps it in a register and the effect disappears) */
volatile long adjacent[MAX_THREADS];              /* 8 counters share one cache line */

typedef struct {
    volatile long value;
    char pad[CACHE_LINE - sizeof(long)];           /* push the next counter to a new line */
} padded_counter;

padded_counter padded[MAX_THREADS] __attribute__((aligned(CACHE_LINE)));

long results[MAX_THREADS];      /* local-accumulate version writes its result here once */
long expected[MAX_THREADS];

/* every thread walks its own random sequence and counts how often bit 16 is set,
   so the three versions do exactly the same amount of work */
static inline unsigned int next_rand(unsigned int x)
{
    return x * 1103515245u + 12345u;
}

void adjacent_version()
{
    #pragma omp parallel
```

```
    {
        int tid = omp_get_thread_num();
        unsigned int x = tid + 1;
        adjacent[tid] = 0;
        for (long i = 0; i < ITERS; i++) {
            x = next_rand(x);
            adjacent[tid] += (x >> 16) & 1;
        }
    }
}

void padded_version()
{
    #pragma omp parallel
    {
        int tid = omp_get_thread_num();
        unsigned int x = tid + 1;
        padded[tid].value = 0;
        for (long i = 0; i < ITERS; i++) {
            x = next_rand(x);
            padded[tid].value += (x >> 16) & 1;
        }
    }
}

void local_version()
{
    #pragma omp parallel
    {
        int tid = omp_get_thread_num();
        unsigned int x = tid + 1;
        long count = 0;                 /* private, lives in a register */
        for (long i = 0; i < ITERS; i++) {
            x = next_rand(x);
            count += (x >> 16) & 1;
        }
        results[tid] = count;           /* one shared write per thread */
    }
}

/* runs a version REPS times, returns the average time in ms */
double time_version(void (*fn)())
{
    double total = 0.0;
    for (int r = 0; r < REPS; r++) {
        double t0 = omp_get_wtime();
        fn();
        total += omp_get_wtime() - t0;
    }
    return total / REPS * 1000.0;
}

int main()
{
    int nthreads = omp_get_max_threads();

    /* serial reference value for every thread's counter */
    for (int t = 0; t < nthreads; t++) {
        unsigned int x = t + 1;
        long count = 0;
        for (long i = 0; i < ITERS; i++) {
            x = next_rand(x);
            count += (x >> 16) & 1;
        }
        expected[t] = count;
    }

    double t_adj = time_version(adjacent_version);
    double t_pad = time_version(padded_version);
    double t_loc = time_version(local_version);

    int ok = 1;
    for (int t = 0; t < nthreads; t++)
        if (adjacent[t] != expected[t] || padded[t].value != expected[t] || results[t] != expected[t])
            ok = 0;

    printf("threads=%d  iters/thread=%ld  sizeof(padded_counter)=%zu bytes\n",
           nthreads, ITERS, sizeof(padded_counter));
    printf("adjacent         %9.2f ms\n", t_adj);
    printf("padded           %9.2f ms\n", t_pad);
    printf("local-accumulate %9.2f ms\n", t_loc);
    printf("slowdown adjacent/padded = %.2fx   counters correct=%s\n",
           t_adj / t_pad, ok ? "YES" : "NO");
    return 0;
```

```
}
```

Task 4: Scheduling Policy Summary

The uniform version is the same program built with `-DUNIFORM`: every iteration costs the average cost of the uneven case, so the total work is the same (serial 771.28 ms). Both workloads use 4 threads, and the uneven numbers are copied from Task 2.

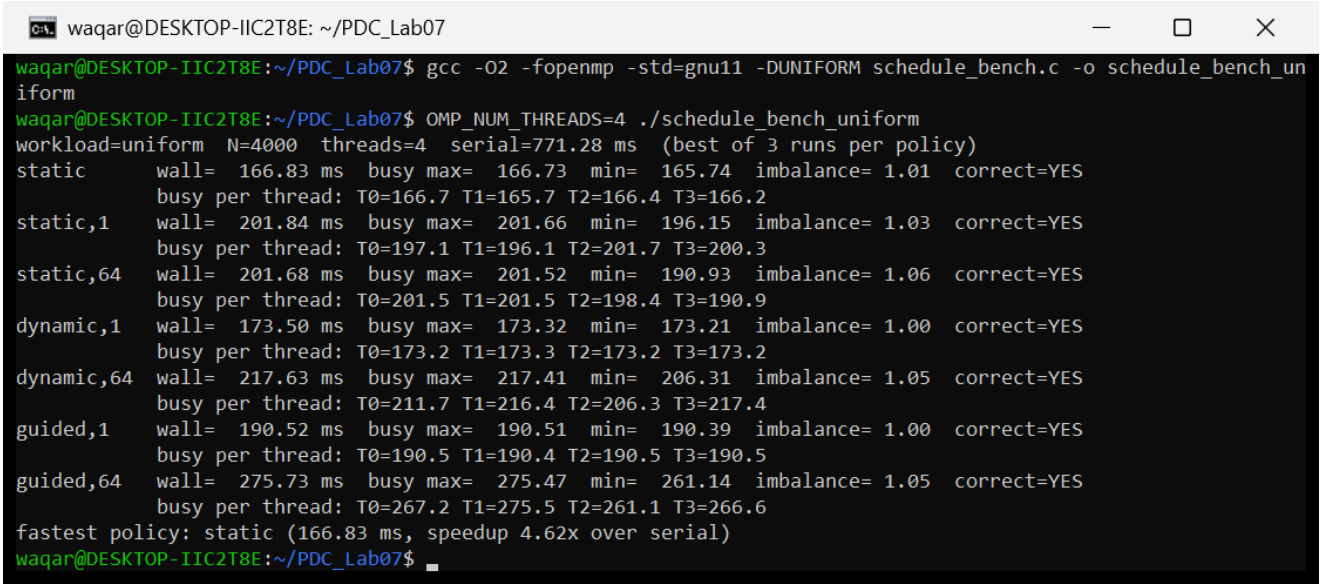
Summary of the two workloads (4 threads)

Policy	Uneven: wall (ms)	Uneven: imbalance	Uniform: wall (ms)	Uniform: imbalance	Verdict
static	292.04	7.18	166.83	1.01	Worst for uneven; **best for uniform**
static,1	152.86	1.01	201.84	1.03	**Best for uneven**; middle for uniform
static,64	257.99	1.08	201.68	1.06	Balanced, middle (uneven run was noisy)
dynamic,1	260.94	1.00	173.50	1.00	Perfect balance; 2nd for uniform
dynamic,64	187.46	1.06	217.63	1.05	Good for uneven, slower for uniform
guided,1	182.19	1.00	190.52	1.00	Good on both, safe choice
guided,64	189.69	1.05	275.73	1.05	Good for uneven; uniform run was slow

Written summary

On the uneven workload, static,1 was the best policy on my machine (152.86 ms, imbalance 1.01), while plain static was clearly the worst (292.04 ms, imbalance 7.18) because the last thread got the most expensive quarter of the loop. The on-demand policies guided,1 (182.19 ms), dynamic,64 (187.46 ms) and guided,64 (189.69 ms) were also close to balanced, which shows that any policy that spreads out the expensive iterations solves the problem. On the uniform workload, plain static was the best (166.83 ms, imbalance 1.01): every block costs the same, so the simplest split is already balanced and has no run-time overhead, and each thread works on a contiguous range. dynamic,1 was a close second (173.50 ms), so the overhead of handing out chunks was small for these fairly expensive iterations. Some rows (static,64 and dynamic,1 uneven, guided,64 uniform) were slowed down by noise on the laptop even though their imbalance was about 1.0, so I base the conclusions on the imbalance as well as the wall time. My rule of thumb: use static when all iterations cost about the same, use dynamic or guided with a moderate chunk when the cost varies or is unknown, and use static,1 when the cost grows smoothly and predictably with the index. Since the loop uses schedule(runtime), it is also easy to measure a few policies with OMP_SCHEDULE before deciding.

Screenshot 11: building the uniform version (-DUNIFORM) and running it with 4 threads.



Discussion questions

Q1. Which policy suited the uneven workload best on your machine, and which suited the uniform workload best? Support each answer with numbers from your table.

Uneven: static,1 with 152.86 ms and imbalance 1.01, against 292.04 ms and imbalance 7.18 for plain static, so it was 1.91x faster than the default. Uniform: plain static with 166.83 ms and imbalance 1.01; the next best was dynamic,1 with 173.50 ms, and the chunked and guided policies were between 190 and 276 ms.

Q2. Why can static be the best or one of the best choices for the uniform workload, even though it was the worst for the uneven one?

Static does no balancing at run time: it just splits the iterations into equal-sized blocks. When every iteration costs the same, equal-sized blocks also mean equal work (busy max 166.73 ms, busy min 165.74 ms, imbalance 1.01), so nothing is gained by balancing. Static then has the lowest overhead (no shared counter, no chunk requests) and the best locality, because each thread walks one contiguous range. On the uneven workload the same equal-sized blocks have very different costs, so it was the worst.

Q3. If you did not know in advance how uneven the iterations are, which policy would you choose as a safe default, and why?

guided (or dynamic with a moderate chunk such as 16). On my machine guided,1 stayed within 10-20% of the best on both workloads (182.19 ms uneven, 190.52 ms uniform, imbalance 1.00 on both) and was never the worst. It adapts at run time like dynamic, but because it starts with large chunks it does not pay much overhead when the work turns out to be uniform. The OMP_SCHEDULE="dynamic,16" run in Task 2 (159 ms, imbalance 1.02) is another good example.

Q4. What would change in your recommendation if each iteration were very cheap (a few nanoseconds), so that claiming a chunk costs as much as the work itself?

Then dynamic,1 and guided,1 near the end of the loop would spend most of their time on the shared counter instead of on the work, and contention on that counter would make them slower than serial. I would use static, or dynamic/guided with a large chunk (hundreds or thousands of iterations), so each chunk claim is paid for by a lot of work. For cheap iterations, reducing overhead matters more than perfect balance.

Q5. How can schedule(runtime) together with OMP_SCHEDULE help you tune a program without recompiling it?

With schedule(runtime) the policy is not fixed in the binary; it is read when the program starts from the OMP_SCHEDULE environment variable (or set with omp_set_schedule). So I can try many policies and chunk sizes from the shell, for example `OMP_SCHEDULE="dynamic,16" ./schedule_bench` or `OMP_SCHEDULE="guided,4"`, on the real machine and the real input, keep the fastest, and even use a different setting on a different machine, all with the same executable.

Lab Completion Checklist

Item	Status
WSL with GCC 15.2 and -fopenmp builds all programs	Done (Screenshots 1, 2, 5, 9, 11)
reduction: sum_ok=YES and max_ok=YES for all versions except racy; scaling table for 1, 2, 4, 8 threads	Done
schedule_bench: correct=YES for all seven policies; uneven workload run with 2, 4 and 8 threads	Done
false_sharing: counters correct=YES; three runs at 4 threads; 1, 2, 4, 8 threads compared	Done
Task 4 summary table and written recommendation with a best policy for each workload	Done
All discussion questions answered; source code, raw output and screenshots included	Done